24K-2016 CY-5A

QNO:1 : Ans:

~~Performance~~

(1) The intelligent agent uses camera & Lidar to percieve the environment e.g. whether the obstacle is a human, rigid material, or just a debris.

PERFORMANCE:

The agent's try to minimize the total risk by getting output from real-time running Machine learning algorithms.

DECISION & ACTION:

Hitting object can potentially injure the passenger. Collision with another car can produce much more worse results and lastly controlled brake can be the expected option.

Performance could be measured through safe driving, environment is the road sensors include camera, Lidar and actuators include controlled brake.

(2) Ans:

Sensing : Camera, LiDAR, Location.

Perception : Object detection, Lane detection traffic signs.

CamScanner CS

State: speed, direction, vehicle position.
Prediction: Predict movement of vehicles, Pedestrians.
Planning & Decision-making: route planning, risk assessment, & action selection.
Action: steering, brake, acceleration.

(3) Ans:

The decision can be justified from the perspectives of rationality & Utility because it maximize the numbers of lives saved. However, it raises significal concerns regarding safety, ethics & responsibility. Therefore, whether the decision is acceptable depends on the values & principles society chooses to prioritize. A balanced approach should seek to minimize harm while ensuring fairness, public trust & accoutable.

(4) Ans:

At this point the model can predict correct output, or it can't predict the correct result, but intelligenke will always fulfile the correct result. AI model should be

CamScanner

24K-2016 CY-5A

adaptive not just ~~pretending~~ Predicting the output from already given scenarios.

(5) Ans:

Sensors are one of the major sources of uncertainty. Bad weather poor lightning conditions & any of sensors failure can lead to wrong decision. Incomplete & incorrect data & also major sources of uncertainty.

Q NO: 2

(1) Ans:

Environment: Doctors, Patients, nurses corridors, wards, rooms, equipment.

Sensors: Camera, LiDAR, Position sensor, Obstacle detection sensors.

Actuators: Wheels/motors, steering mechanism.

Objective: The robot must deliver medicines & other lab sample from pharmacy/lab to destination safely & efficiently while avoiding blockpth.

CamScanner CS

(2) Ans:

State representation:

State = (Pharmacy, carrying Medicine, corridor-2)

→ This representation is sufficient due to:

- Robots current position
- Whether delivery is still pending.
- Information about blocked paths.
- Robot can decide where to move.
- Avoid known blocked corridors.
- Determine whether the goal has been reached.

→ Therefore it contains enough info for decision-making.

Initial State

location = (Pharmacy, carrying Med. No)

Action:

Move Forward, Backward, left, right wait & update map.

Transition model:

It defines the result of an action:

(corridor A, move Right) → corridor B

(corridor B, Detect Blockage) → corridor B + update map

CamScanner

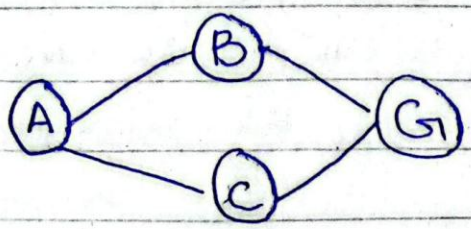
24K-2016 CY-5A

Goal State:
Robot is at the target ward & medicine has been delivered.

Path cost:
Pathcost = Distance + Time + Risk pen.

Solution:
Pharmacy → corridor A → corridor C → corridor E → Target.

(3) Ans:
BFS gives a route with minimum number of steps, it does not guarantee the cost-efficient route. The BFS driven solution will only work if there is same cost edge.



BFS: A → B → G1 ; B is dangerous.
Actual best path: A → C → G1

CamScanner

(4) Ans:

Rationality:

Robot X simply finds the shortest path regardless of corridor blockage, but robot Y also considers other factors including likelihood corridor blockage, importance of delivery, so it finds the path that completes the task even though the path can be long.

Uncertainty:

Robot X does not consider the factors that may make the robot to wait for long time like blockage, importance of delivery. Robot Y could predict the likelihood of an event like blockage, importance of delivery so it finds the path that completes the task even though the path can be long

Adaptation:

Robot X whenever encounters unexpected situations like corridor-blockage it simply waits while Robot Y pause recalculate the possible alternatives & start executing.

CamScanner CS

2410-2016 CY-5A

Performance Measurement:
For robot X finding shortest path is the main goal while Robot Y knows that making more deliveries is more important than being cost effective

(5) A. Disagree, An agent does not need complete info to behave rationally. It can choose the best action based on available info.

B. Disagree, This is explicitly imp. in hospital scenario because dangerous or blocked areas should be avoided even if they provide a shorter route. Rational path ≠ Shortest path.

C. Agree, Both may have different information, sensors observations, internal models, prediction or experience.

D. Disagree, bad outcome ≠ irrational deci. rationality is judged based on the information available when the decision was made, not just by looking at the final result.

CamScanner CS

E. Agree, Sometimes moving away from the goal temporarily allows the agent to reach it more safely successfully.

F. Disagree, lowest immediate cost does not mean lowest total cost.

G. Agree, The hospital robot's digital map may not always reflect the current situation, yet it can make rational decision.

Q:3:

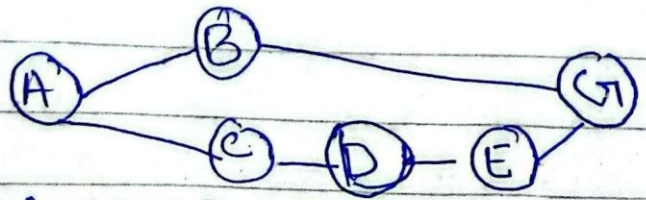
a) Ans: NO, the stmt is not completely correct

Completeness:

Dfs lacks completeness in some cases eg. in large graphs.

Optimal

Dfs does not alway produce optimal sol. There can be a more optimal in case dfs does not find that one.



DFS: A → C → D → E → G

Optimal: A → B → G

CamScanner

24K-2018 CY-5A

Memory:

→ This part of the claim is generally advantage of DFS.

→ DFS follows one branch completely deep before backtracking, rather than storing an entire level like BFS.

~~space com~~

→ Space complexity : $O(bm)$

b: branch factor, m: max depth.

b)

Finite Tree:

In finite tree dfs always produces a result which shows completeness & the result can be either optimal or not optimal.

complete : Yes , Optimal : No

Infinite Search tree:

Complete : NO , Optimal : NO

Dfs doesn't guarantee that it could produce a result and the generated result may or may not be the optimal.

Graphs containing cycles:

Dfs can get caught in infinite loop

CamScanner

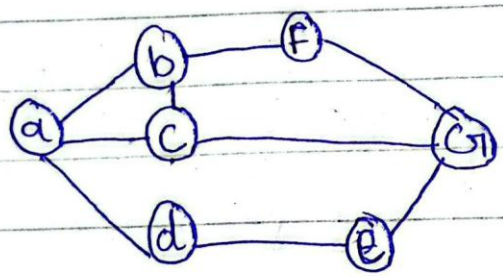
visited nodes were not tracked.
Otherwise in this scenario, limited nodes, the dfs is complete, but it may not produce optimal result.

→ without cycle detection: Not necessarily complete - it can loop.

→ with visited-node tracking in a finite graph: complete.

→ Optimal: NO.

c)



Stack

a
d
c
b
e
F
G

Visited: | a | b | F | G |

Path returned by DFS:

a → b → F → G = 3

Actual shortest:

a → c → g = 2

CamScanner

d)

DFS follows a particular approach so solve a particular problem. Having information about whole graph does not help to change the approach. DFS always select node according to pre-defined rule. DFS does not care about the cost associated with that goal destination.

CamScanner CS

Question 4:

A

BFS:

```
#-----------BFS - START ------------------------
def bfs(maze, start, goal):

    frontier = [(start, [start])]
    visited = {start}

    expanded = 0

    while frontier:

        current, path = frontier.pop(0)

        if current == goal:
            return path, len(path) - 1, expanded, True

        expanded += 1

        row, col = current

        neighbors = [
            (row + 1, col),  # Down
            (row, col + 1),  # Right
            (row - 1, col),  # Up
            (row, col - 1)   # Left
        ]
```

CamScanner

```
        for nr, nc in neighbors:

            if 0 <= nr < len(maze) and 0 <= nc < len(maze[0]):

                if maze[nr][nc] != 1 and (nr, nc) not in visited:

                    visited.add((nr, nc))

                    frontier.append(
                        ((nr, nc), path + [(nr, nc)])
                    )

    return [], 0, expanded, False


def find_start_goal(maze):

    start = None
    goal = None

    for r in range(len(maze)):
        for c in range(len(maze[0])):

            if maze[r][c] == 'S':
                start = (r, c)

            elif maze[r][c] == 'G':
                goal = (r, c)

    return start, goal
```

CamScanner

```
#----------BFS - END ------------------------

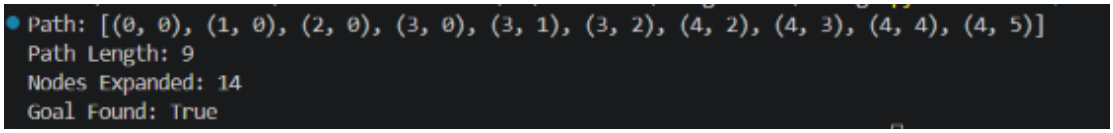

if __name__ == '__main__':
   maze = [
         ['S', 0, 1, 0, 0, 0],
         [0,   0, 1, 0, 1, 0],
         [0,   1, 0, 0, 1, 0],
         [0,   0, 0, 1, 0, 0],
         [1,   1, 0, 0, 0, 'G']
         ]


start, goal = find_start_goal(maze)


path, length, expanded, found = bfs(maze, start, goal)


print("Path:", path)
print("Path Length:", length)
print("Nodes Expanded:", expanded)
print("Goal Found:", found)


#----------BFS - END ------------------------
```



CamScanner

DLS:

```
def dls(maze, start, goal, limit):

    expanded = 0

    def search(current, path, depth):

        nonlocal expanded

        if current == goal:
            return path

        if depth == limit:
            return None

        expanded += 1

        row, col = current

        neighbors = [
            (row + 1, col),  # Down
            (row, col + 1),  # Right
            (row - 1, col),  # Up
            (row, col - 1)   # Left
        ]

        for nr, nc in neighbors:

            if 0 <= nr < len(maze) and 0 <= nc < len(maze[0]):
```

CamScanner

```
            if maze[nr][nc] != 1 and (nr, nc) not in path:

                result = search(
                    (nr, nc),
                    path + [(nr, nc)],
                    depth + 1
                )

                if result is not None:
                    return result

        return None

    path = search(start, [start], 0)

    if path is not None:
        return path, len(path) - 1, expanded, True

    return [], 0, expanded, False

if __name__ == '__main__':
    maze = [
        ['S', 0, 1, 0, 0, 0],
        [0,   0, 1, 0, 1, 0],
        [0,   1, 0, 0, 1, 0],
        [0,   0, 0, 1, 0, 0],
        [1,   1, 0, 0, 0, 'G']
        ]
```

CamScanner

```
start = (0, 0)
goal = (4, 5)

limits = [2, 4, 6, 9]

for limit in limits:

    path, length, expanded, found = dls(
        maze, start, goal, limit
    )

print("\nDepth Limit:", limit)
print("Goal Found:", found)
print("Path:", path)
print("Path Length:", length)
print("Nodes Expanded:", expanded)
```

```
Depth Limit: 9
Goal Found: True
Path: [(0, 0), (1, 0), (2, 0), (3, 0), (3, 1), (3, 2), (4, 2), (4, 3), (4, 4), (4, 5)]
Path Length: 9
Nodes Expanded: 9
```

CamScanner

IDDFS:

```
def dls(maze, start, goal, limit):

    expanded = 0

    def search(current, path, depth):

        nonlocal expanded

        if current == goal:
            return path

        if depth == limit:
            return None

        expanded += 1

        row, col = current

        neighbors = [
            (row + 1, col),  # Down
            (row, col + 1),  # Right
            (row - 1, col),  # Up
            (row, col - 1)   # Left
        ]

        for nr, nc in neighbors:

            if 0 <= nr < len(maze) and 0 <= nc < len(maze[0]):
```

CamScanner

```
            if maze[nr][nc] != 1 and (nr, nc) not in path:

                result = search(
                    (nr, nc),
                    path + [(nr, nc)],
                    depth + 1
                )

                if result is not None:
                    return result

        return None

    path = search(start, [start], 0)

    if path is not None:
        return path, len(path) - 1, expanded, True

    return [], 0, expanded, False

def ids(maze, start, goal):

    limit = 0
    total_expanded = 0

    while True:

        path, length, expanded, found = dls(
            maze, start, goal, limit
```

CamScanner

```
        )

        total_expanded += expanded

        print(
            "Limit:", limit,
            "Expanded:", expanded,
            "Found:", found
        )

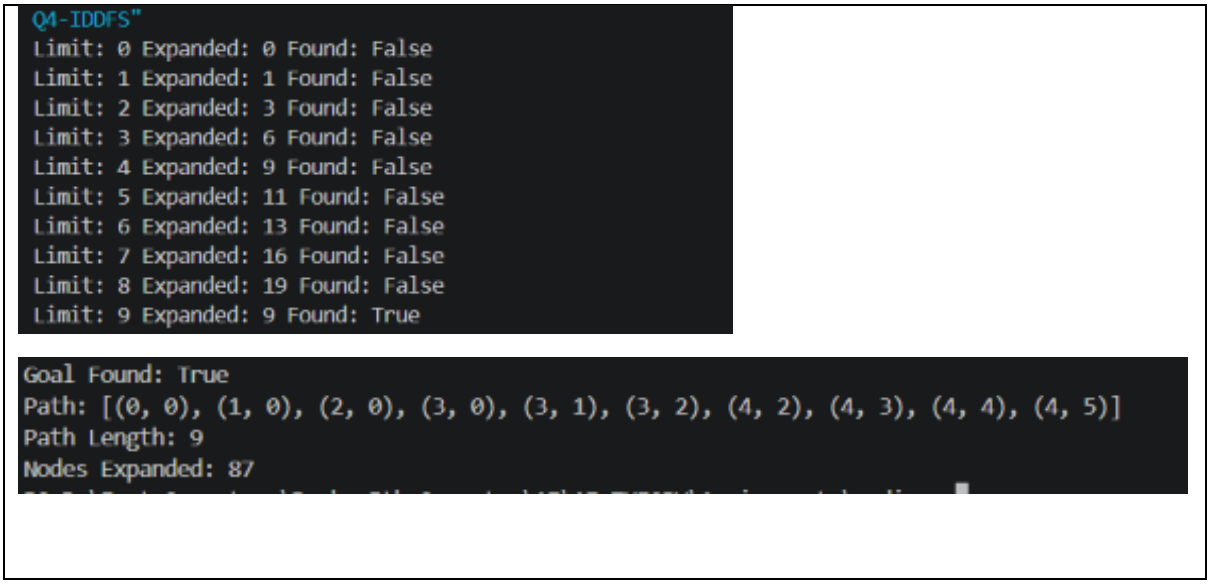
        if found:
            return path, length, total_expanded, True

        limit += 1

if __name__ == '__main__':
    maze = [
            ['S', 0, 1, 0, 0, 0],
            [0,   0, 1, 0, 1, 0],
            [0,   1, 0, 0, 1, 0],
            [0,   0, 0, 1, 0, 0],
            [1,   1, 0, 0, 0, 'G']
            ]
    start = (0, 0)
    goal = (4, 5)

    ids(maze,start,goal)
```

CamScanner



B

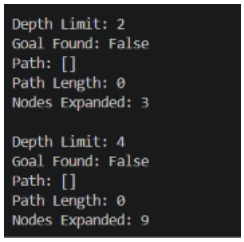
Comparison table:

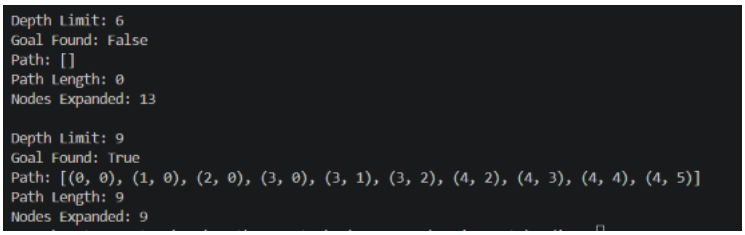
Algorithm	Path Found	Path Length	Nodes Expanded	Goal Found?
BFS	(0, 0), (1, 0), (2, 0), (3, 0), (3, 1), (3, 2), (4, 2), (4, 3), (4, 4), (4, 5)	9	9	Yes
DLS (9)	(0, 0), (1, 0), (2, 0), (3, 0), (3, 1), (3, 2), (4, 2), (4, 3), (4, 4), (4, 5)	9	9	Yes
IDS	(0, 0), (1, 0), (2, 0), (3, 0), (3, 1), (3, 2), (4, 2), (4, 3), (4, 4), (4, 5)	9	87	Yes/No

DLS Table:

Limit	Goal Found?	Path	Path Length	Nodes Expanded
2	No	0	0	output
4	No	0	0	output
6	No	0	0	output
9	Yes	(0, 0), (1, 0), (2, 0), (3, 0), (3, 1), (3, 2), (4, 2), (4, 3), (4, 4), (4, 5)	9	9

CamScanner





C

1:

BFS and IDS found the shortest path in this maze. BFS guarantees the shortest path because it explores nodes level by level. Since every move in the maze has the same cost, the first path BFS finds to the goal will have the minimum number of moves. IDS also finds the shortest-depth solution because it starts from depth 0 and gradually increases the depth limit until the goal is found. In this maze, the shortest path has a length of **9 moves**, as stated in the assignment

2:

DLS is expected to use the least memory. It works like DFS but stops when it reaches the given depth limit. It mainly needs to keep track of the current search path instead of storing all the nodes at a level like BFS does.

IDS also has low memory usage because each individual iteration uses DLS. However, it repeats the search with different depth limits. BFS usually requires more memory because its frontier may contain many nodes from the same level at once.

3:

DLS can fail simply because its **depth limit is too small**.

For example, the goal in this maze requires 9 moves. If I run DLS with:

Limit = 6

it is not allowed to search beyond depth 6. Therefore, it cannot reach the goal even though a valid path exists at depth 9. This is why the assignment states that limits 2, 4, and 6 are insufficient.

4:

IDS solves this problem by **not depending on one fixed depth limit**.

It starts with:

0 → 1 → 2 → 3 → ... → 9

If the goal is not found at one depth, IDS increases the limit and searches again. So even if we don't initially know how deep the goal is, IDS can eventually reach it by gradually increasing the search depth. This is exactly the behavior required by the assignment.

CamScanner

5:

IDS expands some nodes multiple times because every new depth limit starts a new DLS search from the starting node.

For example, the start node can be visited during:

Limit 0

Limit 1

Limit 2

Limit 3

...

Limit 9

So nodes close to the start are searched repeatedly.

This creates some extra work, but it is usually an acceptable trade-off because IDS gets the low memory advantage of depth-first searching while still being able to find a shallowest solution like BFS when step costs are equal.

6:

Property	**BFS**	**DLS**	**IDS**
Time	$O(b^d)$	$O(b^l)$	$O(b^d)$
Space	$O(b^d)$	$O(bl)$	$O(bd)$
Complete	Yes	Not always	Yes
Optimal	Yes, for equal costs	No	Yes, for equal costs

7:

If the goal is moved deeper, **BFS will have to explore many more nodes level by level before reaching it**, so both its running time and memory usage can increase significantly.

For **DLS**, the result depends on the selected limit. If the new goal is deeper than the limit, DLS will fail. We would have to increase the limit.

IDS will continue increasing its depth limit until it reaches the new goal. It will repeat more searches, so it will take more time, but its memory requirement will remain relatively low.

8:

CamScanner

For this maze, I would prefer **BFS** because the maze is small, every move has the same cost, and the goal is only at depth 9. BFS guarantees that the path it returns is a shortest path, and memory is not a major problem for such a small maze.

IDS would also be a reasonable choice if memory were more limited. DLS would be less suitable when we do not know the correct depth limit in advance.

The choice therefore depends on **solution depth, branching factor, and available memory**, rather than one algorithm always being better.

9:

For a **100 × 100 maze**, my recommendation would start to change.

BFS is still complete and optimal for this unweighted maze, but its main disadvantage is memory because it has to maintain a potentially large frontier. As the search space and solution depth increase, this becomes more important.

DLS uses less memory, but choosing a suitable depth limit becomes difficult. If the limit is below the actual solution depth, it will fail even when a solution exists.

IDS would become more attractive because it is complete for this type of finite search space, can find a shortest-depth solution with equal step costs, and uses much less memory than BFS. Its disadvantage is that it repeatedly searches nodes near the start, but this can be an acceptable trade-off when memory is limited.

10:

For a **large emergency-navigation problem among these three algorithms, I would choose IDS when memory is an important constraint**. BFS is attractive because it guarantees the shortest path in an unweighted maze, but its memory requirements can become large as the search space grows. DLS saves memory but can fail if the chosen depth limit is too small.

IDS provides a useful balance. It gradually increases the search depth, so we do not need to know the solution depth beforehand. It also keeps the lower memory requirements of depth-first searching while still finding a shallowest solution when all moves have equal cost.

Therefore, for a small maze I would prefer **BFS for its simplicity and shortest-path guarantee**, while for a much larger maze with limited memory I would prefer **IDS among the three algorithms given in the assignment**.

CamScanner